

Parte 4

1. Configura `springdoc-openapi` en el proyecto.

```
<dependencies>
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.6.0</version>
  </dependency>
</dependencies>
```

```
@Configuration
public class OpenApiConfig {

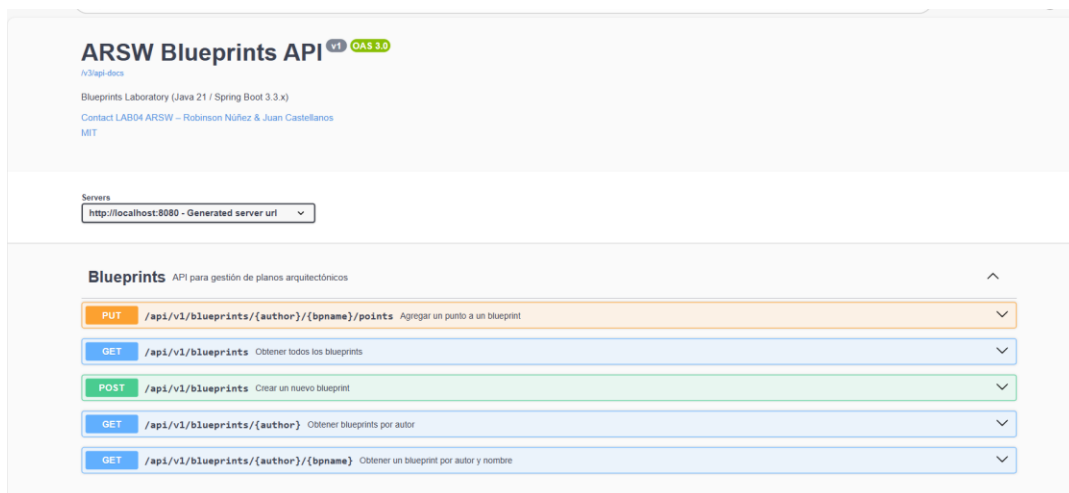
    @Bean
    public OpenAPI api() {
        return new OpenAPI().info(new Info()
            .title(title: "ARSW Blueprints API")
            .version(version: "v1")
            .description(description: "Blueprints Laboratory (Java 21 / Spring Boot 3.3.x)")
            .contact(new Contact()
                .name(name: "LAB04 ARSW | Robinson Núñez & Juan Castellanos")
                .email(email: "robinson.nunez-p@mail.escuelaing.edu.co | juan.ccastellanos@mail.escuelaing.edu.co"))
            .license(new License().name(name: "MIT").url(url: "https://opensource.org/licenses/MIT")));
    }
}
```

Respuesta:

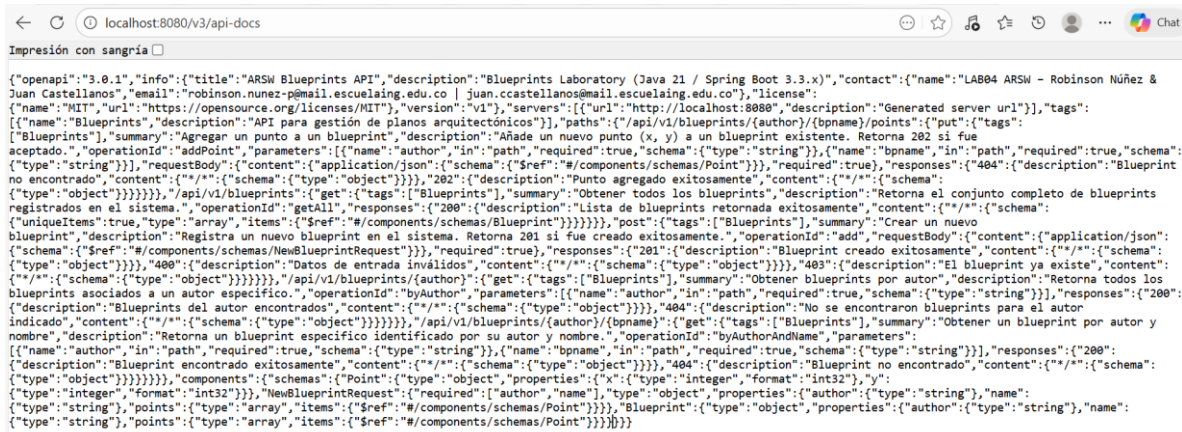
Se reviso que estuviera la dependencia springdoc-openapi-starter-webmvc-ui 2.6.0 en el pom.xml y se le agrego a la clase **OpenAPI** nuestros nombres de autores y la licencia MIT.

2. Expón documentación automática en `/swagger-ui.html`.

- Interfaz de Swagger con las descripciones y configuración del OpenAPI



- Archivo de especificación en formato JSON



- // GET /api/v1/blueprints con la descripción del response

Code	Description
200	Lista de blueprints retornada exitosamente

Media type

▼

Controls Accept header.

- // POST /api/v1/blueprints con la descripción de los response

Code	Description	Links
201	Blueprint creado exitosamente Media type */* Controls Accept header Example Value Schema <pre>{}</pre>	No links
400	Datos de entrada inválidos Media type */* Example Value Schema <pre>()</pre>	No links
403	El blueprint ya existe Media type */* Example Value Schema <pre>()</pre>	No links

- // GET /api/v1/blueprints/{author} con la descripción de los response

Code	Description	Links
200	Blueprints del autor encontrados Media type */* Controls Accept header. Example Value Schema {}	No links
404	No se encontraron blueprints para el autor indicado Media type */* Example Value Schema {}	No links

- // GET /api/v1/blueprints/{author}/{bpname} con la descripción de los response

Code	Description	Links
200	Blueprint encontrado exitosamente Media type */* Controls Accept header. Example Value Schema {}	No links
404	Blueprint no encontrado Media type */* Example Value Schema {}	No links

- // PUT /api/v1/blueprints/{author}/{bpname}/points con la descripción de los response

Code	Description	Links
202	Punto agregado exitosamente Media type */* Controls Accept header. Example Value Schema {}	No links
404	Blueprint no encontrado Media type */* Example Value Schema {}	No links

Respuesta:

Con solo tener la dependencia y **OpenApiConfig**, Spring Boot expone automáticamente la interfaz visual de Swagger en <http://localhost:8080/swagger-ui.html> y el JSON de la

especificación en <http://localhost:8080/v3/api-docs>, rutas configuradas en `application.properties`.

3. "Anota endpoints con `@Operation` y `@ApiResponse`

```
@RestController
//...
@Api(name = "Blueprints", description = "API para gestión de planos arquitectónicos")
public class BlueprintsApiController {

    private final BlueprintsServices services;

    public BlueprintsApiController(BlueprintsServices services) { this.services = services; }

    // GET /api/v1/blueprints
    @Operation(summary = "Obtener todos los blueprints", description = "Retorna el conjunto completo de blueprints registrados en el sistema.")
    @ApiResponse(responseCode = "200", description = "Lista de blueprints retornada exitosamente")
    @GetMapping
    public ResponseEntity<Set<Blueprint>> getAll() {
        return ResponseEntity.ok(services.getAllBlueprints());
    }

    // GET /api/v1/blueprints/{author}
    @Operation(summary = "Obtener blueprints por autor", description = "Retorna todos los blueprints asociados a un autor específico.")
    @ApiResponse(responseCode = "200", description = "Blueprints del autor encontrados"),
    @ApiResponse(responseCode = "404", description = "No se encontraron blueprints para el autor indicado")
    @GetMapping("/{author}")
    public ResponseEntity<> byAuthor(@PathVariable String author) {
        try {
            return ResponseEntity.ok(services.getBlueprintsByAuthor(author));
        } catch (BlueprintNotFoundException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(k1: "error", e.getMessage()));
        }
    }

    // GET /api/v1/blueprints/{author}/{bpname}
    @Operation(summary = "Obtener un blueprint por autor y nombre", description = "Retorna un blueprint específico identificado por su autor y nombre.")
    @ApiResponse(responseCode = "200", description = "Blueprint encontrado exitosamente"),
    @ApiResponse(responseCode = "404", description = "Blueprint no encontrado")
    @GetMapping("/{author}/{bpname}")
    public ResponseEntity<> byAuthorAndName(@PathVariable String author, @PathVariable String bpname) {
        try {
            return ResponseEntity.ok(services.getBlueprint(author, bpname));
        } catch (BlueprintNotFoundException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(k1: "error", e.getMessage()));
        }
    }
}
```

```

// POST /api/v1/blueprints
@Operation(summary = "Crear un nuevo blueprint", description = "Registra un nuevo blueprint en el sistema. Retorna 201 si fue creado exitosamente.")
@ApiResponses({
    @ApiResponse(responseCode = "201", description = "Blueprint creado exitosamente"),
    @ApiResponse(responseCode = "400", description = "Datos de entrada inválidos"),
    @ApiResponse(responseCode = "403", description = "El blueprint ya existe")
})
@PostMapping
public ResponseEntity<?> add(@Valid @RequestBody NewBlueprintRequest req) {
    try {
        Blueprint bp = new Blueprint(req.author(), req.name(), req.points());
        services.addNewBlueprint(bp);
        return ResponseEntity.status(HttpStatus.CREATED).build();
    } catch (BlueprintPersistenceException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(k1: "error", e.getMessage()));
    }
}

// PUT /api/v1/blueprints/{author}/{bpname}/points
@Operation(summary = "Agregar un punto a un blueprint", description = "Añade un nuevo punto (x, y) a un blueprint existente. Retorna 202 si fue aceptado.")
@ApiResponses({
    @ApiResponse(responseCode = "202", description = "Punto agregado exitosamente"),
    @ApiResponse(responseCode = "404", description = "Blueprint no encontrado")
})
@PutMapping("/{author}/{bpname}/points")
public ResponseEntity<?> addPoint(@PathVariable String author,
    @PathVariable String bpname,
    @RequestBody Point p) {
    try {
        services.addPoint(author, bpname, p.x(), p.y());
        return ResponseEntity.status(HttpStatus.ACCEPTED).build();
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(k1: "error", e.getMessage()));
    }
}

public record NewBlueprintRequest(
    @NotBlank String author,
    @NotBlank String name,
    @Valid java.util.List<Point> points
) {}

```

Respuesta:

En **BlueprintsAPIController.java** se anotó cada endpoint con:

- **@Tag** en la clase para agrupar todos los endpoints bajo la sección Blueprints.
- **@Operation** en cada método con summary y description explicando qué hace.
- **@ApiResponses** y **@ApiResponse** documentando los posibles códigos HTTP 200, 201, 202, 400, 403, 404 con su descripción

Esto permite que cualquier consumidor de la API entienda su comportamiento directamente desde la interfaz de Swagger sin necesidad de leer el código.